Analysis and Design of Algorithms
(**BE04000241**)



VVP ENGINEERING COLLEGE

name	**Chauhan tanish**
Enrollment no.	**250473107003**
Subject	**ADA (BE04000241)**
div	**4th CE GZ3**

V.V.P. Engineering College, Rajkot

Vision of the Institute

To be an exemplary institute, transforming students into competent professionals with human values.

Mission of the Institute

1. To provide a conducive academic environment for strengthening technical capabilities of the students.
2. To strengthen linkages with industries, alumni and professional bodies.
3. To organize various co-curricular and extra-curricular activities for overall development of the students.
4. To practice good governance and conduct value- based activities for making students responsible citizens.



V.V.P. ENGINEERING COLLEGE, RAJKOT

Department of Computer Engineering

Vision of the Department

Transforming students into globally efficient professionals with moral values.

Mission of the Department

- To provide a strong foundation of computer engineering through effective teaching learning process.
- To enhance industry linkage & alumni network for better placement and real-world exposure.
- To provide various opportunities & platforms for all round development of students & encourage them for value-based practices.



Department of Computer Engineering

Program Educational Objectives (PEOs)

Graduates will be able to

- Apply computer engineering theories, principles and skills to meet the challenges of the society.
- Communicate effectively, work collaboratively and manifest professionalism with ethics.
- Exhibit life-long learning attitude and adapt to rapid technological changes in industry.
- Advance their career in industry, pursue higher education or become an entrepreneur.

Program Specific Outcomes (PSOs)

Graduates will be able to

- **PSO1:** Apply fundamental knowledge of hardware and software aspects of computer systems.
- **PSO2:** Analyze, model and develop computer applications by adapting emerging technologies and standard practices of software project development to meet the requirements of industry and society.
- **PSO3:** Use different programming languages and open- source platforms.

Practical 1

Title

Implementation of Bubble Sort

Objective:

To study and implement the Bubble Sort algorithm to arrange elements of an array in ascending or descending order.

Theory:

Bubble Sort is a simple sorting algorithm that repeatedly compares adjacent elements of an array and swaps them if they are in the wrong order. This process is repeated for several passes until the array becomes completely sorted. After each pass, the largest element moves to its correct position at the end of the array, just like a bubble rising to the surface. Bubble Sort is easy to understand and implement, but it is inefficient for large datasets due to its high time complexity.

Algorithm / Steps:

1. Start
2. Read the number of elements n
3. Read the array elements
4. Repeat steps 5 to 8 for i = 0 to n - 2
5. Set swapped = false
6. Repeat steps 7 to 8 for j = 0 to n - i - 2
7. If array[j] > array[j + 1], then
   - Swap array[j] and array[j + 1]
   - Set swapped = true
8. If swapped == false, then
   - Break the loop (array is already sorted)
9. End

Bubble sort code:

```
#include<stdio.h>
#include <stdbool.h>
int main()
{

    int a[] = {14, 3, 19, 7, 2, 11, 6, 18, 1, 10, 5, 15, 9, 4, 13};
    int n = sizeof(a)/sizeof(a[0]);
    int i,j,temp;
    bool swapped;
```

```
    for( i = 0; i < n-1; i++ ){
        swapped = false;
        for( j = 0; j < n-i-1; j++){
            if(a[j] > a[j+1]){

                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;

                swapped = true;
            }
        }

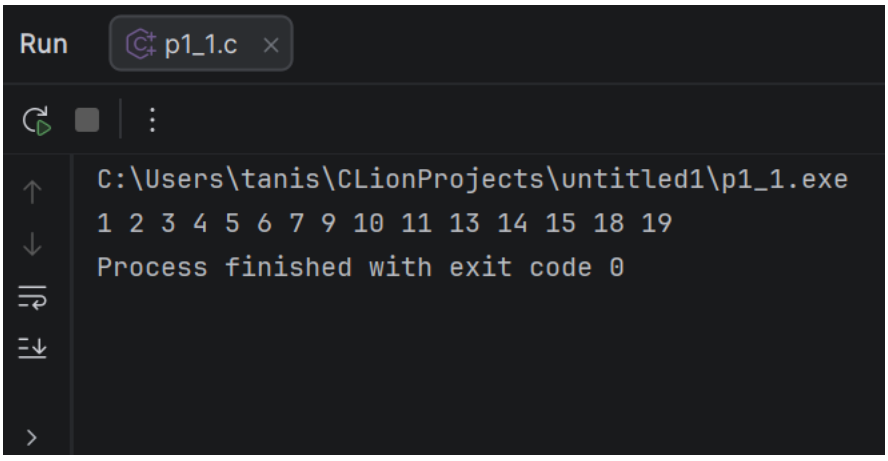
        if(!swapped){
            break;
        }
    }

    for(i = 0; i<n; i++){
        printf("%d ", a[i]);
    }

     return 0 ;

}
```

Output:



Array after 4th pass:

```
#include<stdio.h>
#include <stdbool.h>
int main()
{

    int a[] = {14, 3, 19, 7, 2, 11, 6, 18, 1, 10, 5, 15, 9, 4, 13};
    int n = sizeof(a)/sizeof(a[0]);
    int i,j,temp;
    bool swapped;


    for( i = 0; i < n-1; i++ ){
        swapped = false;
        for( j = 0; j < n-i-1; j++){
            if(a[j] > a[j+1]){

                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;


                swapped = true;
            }
        }

        if(!swapped){
            break;
        }

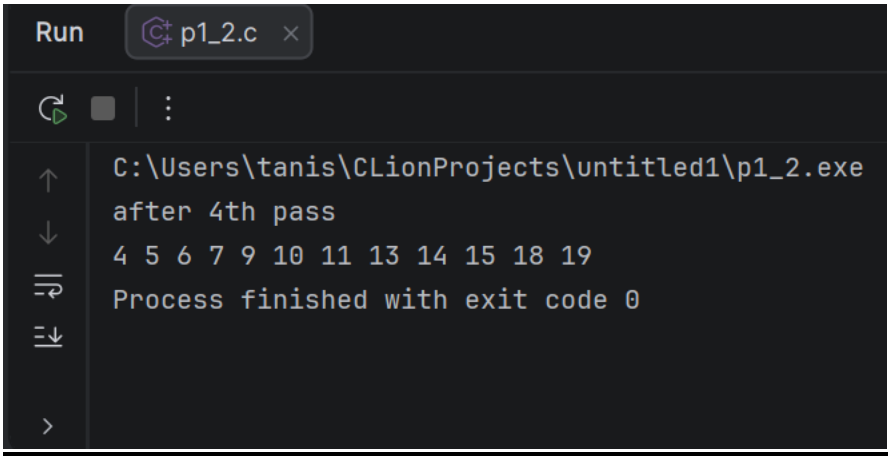
    }
    printf("after 4th pass \n");
    for(i = 3; i<n; i++){

        printf("%d ", a[i]);
    }

     return 0 ;

}
```

Output:


Run
p1_2.c
C:\Users\tanis\CLionProjects\untitled1\p1_2.exe
after 4th pass
4 5 6 7 9 10 11 13 14 15 18 19
Process finished with exit code 0

Array after 7th pass:

```
#include<stdio.h>
#include <stdbool.h>
int main()
{

    int a[] = {14, 3, 19, 7, 2, 11, 6, 18, 1, 10, 5, 15, 9, 4, 13};
    int n = sizeof(a)/sizeof(a[0]);
    int i,j,temp;
    bool swapped;


    for( i = 0; i < n-1; i++ ){
        swapped = false;
        for( j = 0; j < n-i-1; j++){
            if(a[j] > a[j+1]){

                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;

                swapped = true;
            }
        }

        if(!swapped){
            break;
        }

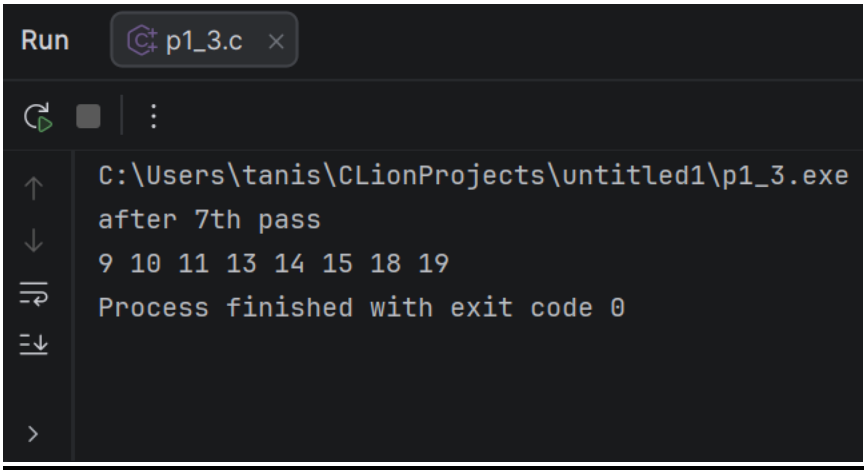
    }
    printf("after 7th pass \n");
    for(i = 7; i<n; i++){

        printf("%d ", a[i]);
    }

     return 0 ;

}
```

Output:


Run
p1_3.c
C:\Users\tanis\CLionProjects\untitled1\p1_3.exe
after 7th pass
9 10 11 13 14 15 18 19
Process finished with exit code 0

Total swaps in pass 5:

```
#include<stdio.h>
#include <stdbool.h>
int main()
{
    int a[] = {14, 3, 19, 7, 2, 11, 6, 18, 1, 10, 5, 15, 9, 4, 13};
    int n = sizeof(a)/sizeof(a[0]);
    int i,j,temp;
    bool swapped;
    int count = 0;
    int swap_pass;

    for( i = 0; i < n-1; i++ ){
        swapped = false;
        count++;
        swap_pass = 0;

        for( j = 0; j < n-i-1; j++){
            if(a[j] > a[j+1]){

                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;

                swapped = true;
                        swap_pass++;

            }
        }

        if(count == 5){
            printf("\ntotal swap in pass 5 : %d\n", swap_pass);
        }

        if(!swapped){
            break;
        }

    }

    for(i = 0; i<n; i++){

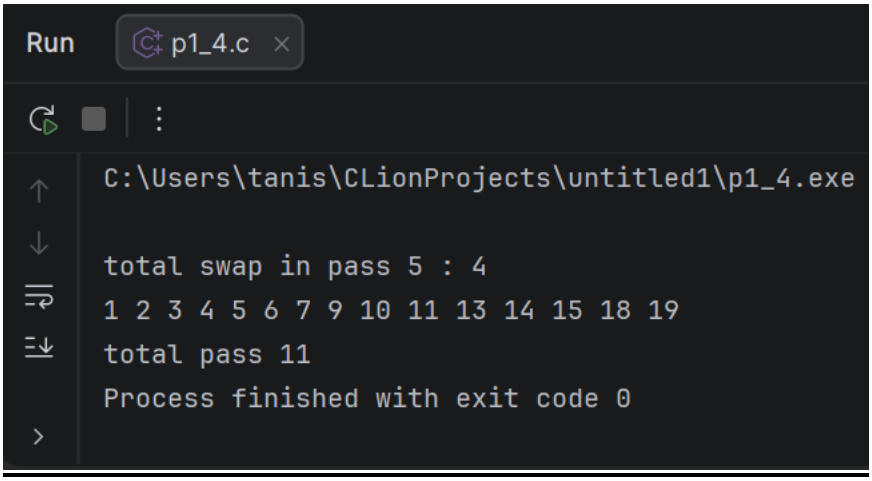
        printf("%d ", a[i]);
    }

    printf("\ntotal pass %d", count);

     return 0 ;

}
```

Output:


Run
p1_4.c
C:\Users\tanis\CLionProjects\untitled1\p1_4.exe
total swap in pass 5 : 4
1 2 3 4 5 6 7 9 10 11 13 14 15 18 19
total pass 11
Process finished with exit code 0

Total swaps for Completely sorted array:

```
#include<stdio.h>
#include <stdbool.h>
int main()
{

int a[] = {14, 3, 19, 7, 2, 11, 6, 18, 1, 10, 5, 15, 9, 4, 13};
    int n = sizeof(a)/sizeof(a[0]);
    int i,j,temp;
    bool swapped;
    int total_swap = 0;

    for( i = 0; i < n-1; i++ ){
        swapped = false;



        for( j = 0; j < n-i-1; j++){
            if(a[j] > a[j+1]){

                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;

                swapped = true;
                total_swap++;

            }
        }



        if(!swapped){
            break;
        }

    }

    for(i = 0; i<n; i++){

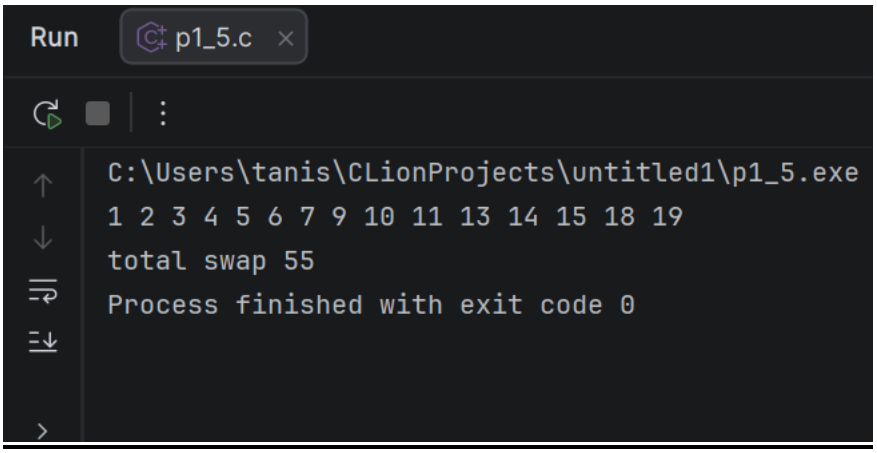
        printf("%d ", a[i]);
    }

    printf("\ntotal swap %d", total_swap);

     return 0 ;

}
```

Output:


Run
p1_5.c
C:\Users\tanis\CLionProjects\untitled1\p1_5.exe
1 2 3 4 5 6 7 9 10 11 13 14 15 18 19
total swap 55
Process finished with exit code 0

Insertion Sort

Objective

Implementation of Insertion Sort

Objective:

To study and implement the Insertion Sort algorithm for arranging elements of an array in ascending or descending order.

Theory:

Insertion Sort is a simple and efficient sorting algorithm that works the way we sort playing cards in our hands. It builds the sorted array one element at a time by taking each element and placing it in its correct position among the previously sorted elements. The algorithm compares the current element with the elements before it and shifts larger elements one position to the right to make space. Insertion Sort is easy to understand and performs well for small or nearly sorted datasets.

Algorithm / Steps:

1. Start
2. Read the number of elements n
3. Read the array elements
4. Repeat steps 5 to 8 for i = 1 to n - 1
5. Set key = array[i]
6. Set j = i - 1
7. While j >= 0 and array[j] > key, do
   - Move array[j] to array[j + 1]
   - Decrease j by 1
8. Insert key at position array[j + 1]
9. End

Code:

```
#include <stdio.h>

int main() {
    int a[15] = {29,10,14,37,13,7,25,18,3,40,21,6,31,12,2};
    int i, j, key;
    int comp = 0, swap = 0;

    for (i = 1; i < 15; i++) {
        key = a[i];
        j = i - 1;
```

```
        while (j >= 0 && a[j] > key) {
            comp++;
            a[j + 1] = a[j];
            swap++;
            j--;
        }
        if (j >= 0)
            comp++;

        a[j + 1] = key;
    }

    printf("Ascending Order:\n");
    for (i = 0; i < 15; i++)
        printf("%d ", a[i]);

    printf("\nTotal Comparisons = %d", comp);
    printf("\nTotal Swaps = %d\n\n", swap);

    comp = 0;
    swap = 0;

    for (i = 1; i < 15; i++) {
        key = a[i];
        j = i - 1;

        while (j >= 0 && a[j] < key) {
            comp++;
            a[j + 1] = a[j];
            swap++;
            j--;
        }
        if (j >= 0)
            comp++;

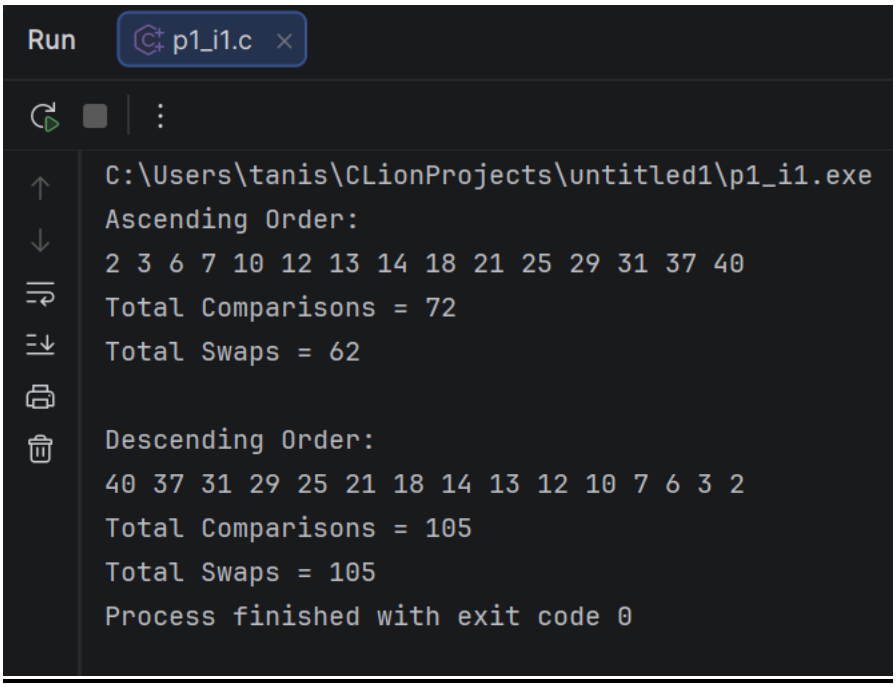
        a[j + 1] = key;
    }

    printf("Descending Order:\n");
    for (i = 0; i < 15; i++)
        printf("%d ", a[i]);

    printf("\nTotal Comparisons = %d", comp);
    printf("\nTotal Swaps = %d", swap);

    return 0;
}
```

Output:


Run
p1_i1.c
C:\Users\tanis\CLionProjects\untitled1\p1_i1.exe
Ascending Order:
2 3 6 7 10 12 13 14 18 21 25 29 31 37 40
Total Comparisons = 72
Total Swaps = 62
Descending Order:
40 37 31 29 25 21 18 14 13 12 10 7 6 3 2
Total Comparisons = 105
Total Swaps = 105
Process finished with exit code 0

Selection Sort

Objective

Implementation of Selection Sort

Objective:

To study and implement the Selection Sort algorithm for arranging elements of an array in ascending or descending order.

Theory:

Selection Sort is a simple comparison-based sorting algorithm. It works by repeatedly selecting the smallest (or largest) element from the unsorted part of the array and placing it at the beginning. The algorithm divides the array into two parts: a sorted part and an unsorted part. In each pass, the minimum element from the unsorted section is found and swapped with the first element of that section. This process continues until the entire array is sorted. Selection Sort is easy to understand but is not efficient for large datasets due to its fixed number of comparisons.

Algorithm / Steps:

1. Start
2. Read the number of elements n
3. Read the array elements
4. Repeat steps 5 to 8 for i = 0 to n - 2
5. Set min = i
6. Repeat step 7 for j = i + 1 to n - 1
7. If array[j] < array[min], then set min = j
8. Swap array[i] and array[min]
9. End

Code:

```
#include <stdio.h>

int main() {
    int a[15] = {29,10,14,37,13,7,25,18,3,40,21,6,31,12,2};
    int i, key, found = 0;
    int comp = 0;

    key = 18;
    found = 0;
```

```
    comp = 0;

    for (i = 0; i < 15; i++) {
        comp++;
        if (a[i] == key) {
            found = 1;
            break;
        }
    }

    if (found)
        printf("Key %d found at position %d\n", key, i + 1);
    else
        printf("Key %d not found\n", key);

    printf("Total Comparisons = %d\n\n", comp);

    key = 16;
    found = 0;
    comp = 0;

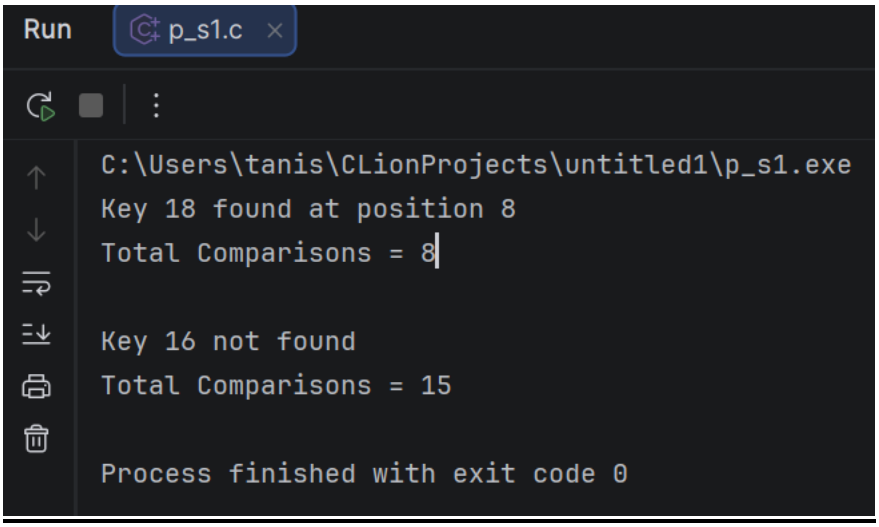
    for (i = 0; i < 15; i++) {
        comp++;
        if (a[i] == key) {
            found = 1;
            break;
        }
    }

    if (found)
        printf("Key %d found at position %d\n", key, i + 1);
    else
        printf("Key %d not found\n", key);

    printf("Total Comparisons = %d\n", comp);

    return 0;
}
```

Output:


Run
p_s1.c
C:\Users\tanis\CLionProjects\untitled1\p_s1.exe
Key 18 found at position 8
Total Comparisons = 8
Key 16 not found
Total Comparisons = 15
Process finished with exit code 0

Practical 2

Title:

Implementation of Binary Search

Objective:

To study and implement the Binary Search technique to efficiently search a given key element in a sorted array.

Theory:

Binary Search is an efficient searching algorithm that works on the principle of divide and conquer. It requires the array to be sorted before searching. In this method, the array is repeatedly divided into two halves, and the middle element is compared with the key element. If the key matches the middle element, the search is successful. If the key is smaller, the search continues in the left half; if it is larger, the search continues in the right half. This process continues until the key is found or the search interval becomes empty. Binary Search is much faster than linear search, especially for large datasets.

Algorithm / Steps:

1. Start
2. Read the number of elements n
3. Read the sorted array elements
4. Read the key element to be searched
5. Set low = 0 and high = n - 1
6. While low ≤ high, repeat steps 7 to 10
7. Set mid = (low + high) / 2
8. If array[mid] == key, then
   - Display “Key Found” and stop
9. Else if key < array[mid], then
   - Set high = mid - 1
10. Else
    - Set low = mid + 1
11. If low > high, display “Key Not Found”
12. End

Code:

```
#include <stdio.h>

int main() {

    int a[15] = {40,37,31,29,25,21,18,14,13,12,10,7,6,3,2};
    int key = 18;
    int low = 0, high = 14, mid;
    int comp = 0, found = 0;

    printf("Descending Sorted Array:\n");
    for (int i = 0; i < 15; i++) {
        printf("%d ", a[i]);
    }

    while (low <= high) {
        comp++;
        mid = (low + high) / 2;

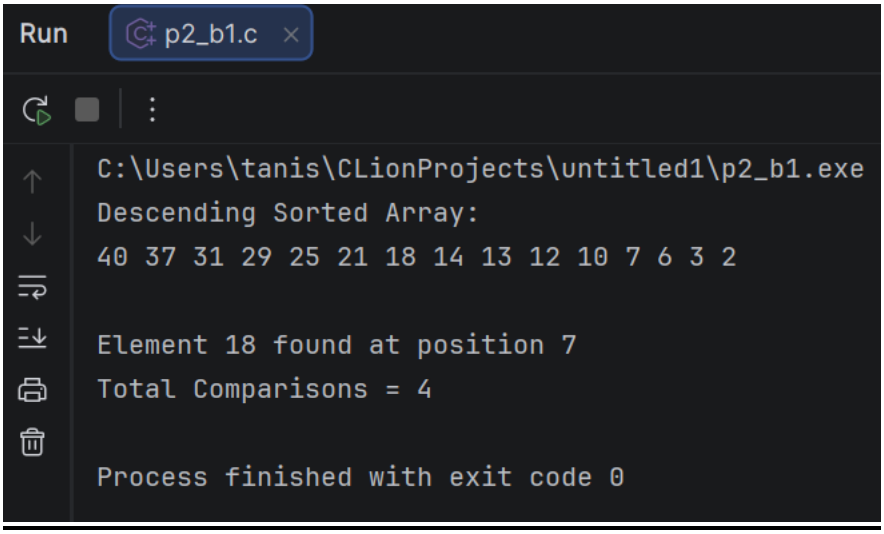
        if (a[mid] == key) {
            printf("\n\nElement %d found at position %d", key, mid + 1);
            found = 1;
            break;
        }
        else if (key > a[mid]) {
            high = mid - 1;   // move left
        }
        else {
            low = mid + 1;    // move right
        }
    }

    if (!found)
        printf("\n\nElement %d not found", key);

    printf("\nTotal Comparisons = %d\n", comp);

    return 0;
}
```

Output:


Run
p2_b1.c
C:\Users\tanis\CLionProjects\untitled1\p2_b1.exe
Descending Sorted Array:
40 37 31 29 25 21 18 14 13 12 10 7 6 3 2
Element 18 found at position 7
Total Comparisons = 4
Process finished with exit code 0

Code:

```
#include <stdio.h>

int main() {
    char ch[26] = "abcdefghijklmnopqrstuvwxyz";
    char key;
    int low = 0, high = 25, mid;

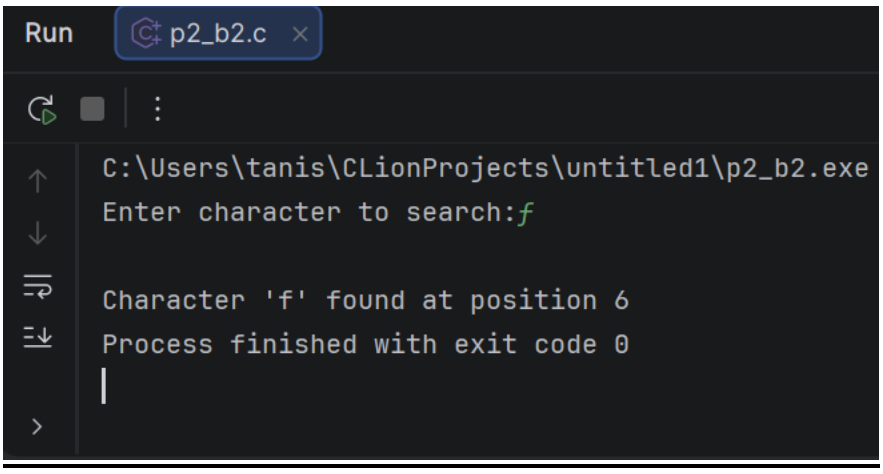
    printf("Enter character to search: ");
    scanf(" %c", &key);

    while (low <= high) {
        mid = (low + high) / 2;

        if (ch[mid] == key) {
            printf("Character '%c' found at position %d", key, mid + 1);
            return 0;
        }
        else if (key < ch[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }

    printf("Character '%c' not found", key);
    return 0;
}
```

Output:



Practical 7

Title:

To solve the Coin Change (Make a Change) Problem using Dynamic Programming.

Objective:

To determine the **minimum number of coins required** to make a given amount using available coin denominations by applying **Dynamic Programming technique**.

Theory:

The Coin Change Problem is a classic problem in Dynamic Programming.
In this problem, we are given a set of coin denominations and a target amount. The goal is to find the minimum number of coins needed to make that amount.

Algorithm / Steps:

1. Start
2. Input number of coin denominations n
3. Input coin values coin[ ]
4. Input target amount A
5. Create an array dp[0…A]
6. Initialize dp[0] = 0
7. For all other values set dp[i] = infinity
8. For each amount i from 1 to A
9. For each coin c
10. If $c \leq i$
11. Update
    dp[i] = min(dp[i], dp[i − c] + 1)
12. The answer will be stored in dp[A]
13. Print minimum number of coins
14. Stop

Code:

```
#include <stdio.h>
#define INF 999

int min(int a,int b)
{
    if(a<b)
        return a;
    return b;
}

int main()
{
    int n,C;

    printf("Enter number of coins: ");
    scanf("%d",&n);

    int coin[n];

    printf("Enter coin values:\n");
    for(int i=0;i<n;i++)
        scanf("%d",&coin[i]);

    printf("Enter capacity (C): ");
    scanf("%d",&C);

    int dp[n+1][C+1];

    for(int i=0;i<=n;i++)
    {
        for(int j=0;j<=C;j++)
        {
            if(j==0)
                dp[i][j]=0;

            else if(i==0)
                dp[i][j]=INF;

            else if(coin[i-1] <= j)
                dp[i][j]=min(1+dp[i][j-coin[i-1]], dp[i-1][j]);

            else
                dp[i][j]=dp[i-1][j];
        }
    }
```

```
    printf("\nDP Table\n\n");

    printf("Coins\\Amt ");
    for(int j=0;j<=C;j++)
        printf("%3d",j);

    printf("\n");

    for(int i=0;i<=n;i++)
    {
        if(i==0)
            printf("0        ");
        else
            printf("%d        ",coin[i-1]);

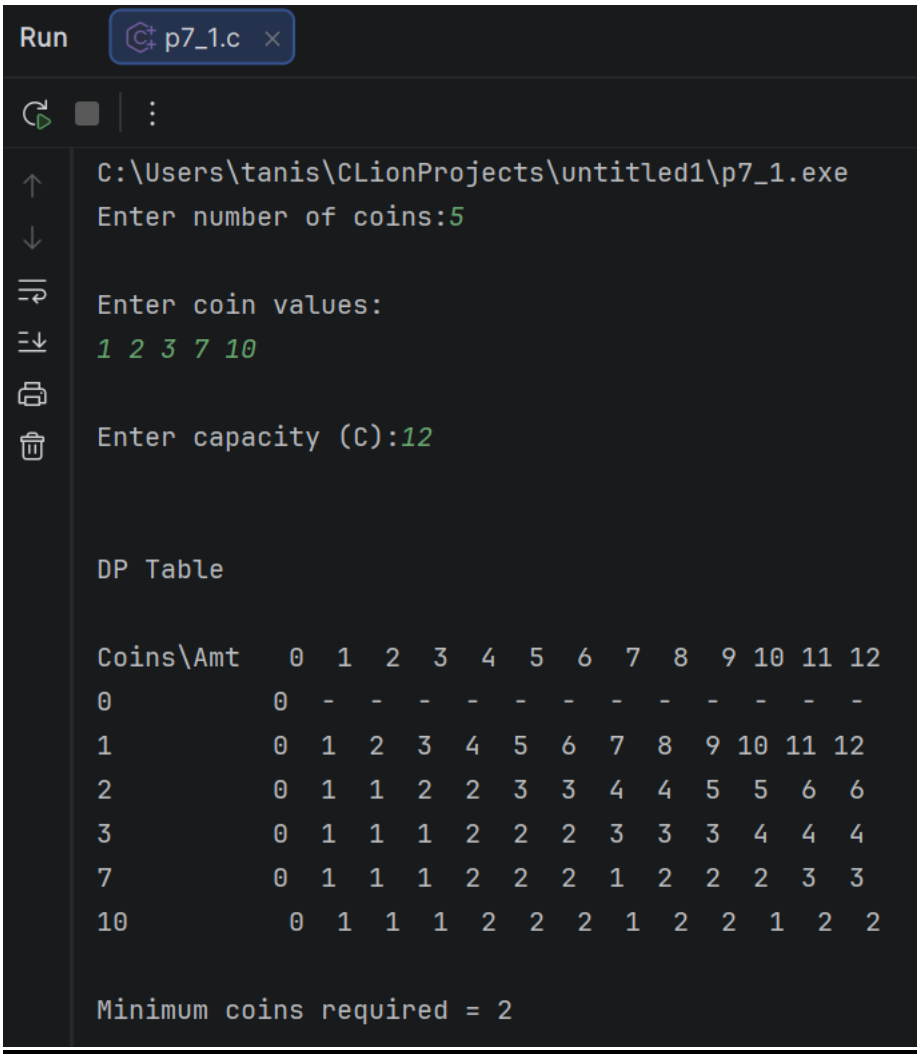
        for(int j=0;j<=C;j++)
        {
            if(dp[i][j]==INF)
                printf("  -");
            else
                printf("%3d",dp[i][j]);
        }

        printf("\n");
    }

    printf("\nMinimum coins required = %d\n",dp[n][C]);

    return 0;
}
```

output:


Run
p7_1.c
C:\Users\tanis\CLionProjects\untitled1\p7_1.exe
Enter number of coins:5
Enter coin values:
1 2 3 7 10
Enter capacity (C):12
DP Table
Coins\Amt 0 1 2 3 4 5 6 7 8 9 10 11 12
0 0 - - - - - - - - - - - -
1 0 1 2 3 4 5 6 7 8 9 10 11 12
2 0 1 1 2 2 3 3 4 4 5 5 6 6
3 0 1 1 1 2 2 2 3 3 3 4 4 4
7 0 1 1 1 2 2 2 1 2 2 2 3 3
10 0 1 1 1 2 2 2 1 2 2 1 2 2
Minimum coins required = 2

Title:

To solve the 0/1 Knapsack Problem using Dynamic Programming.

Objective:

To find the **maximum profit** that can be obtained without exceeding the **capacity of the knapsack** using Dynamic Programming.

Theory:

The 0/1 Knapsack Problem is a fundamental problem in algorithm design.

Algorithm / Steps:

1. Start
2. Input number of items n
3. Input weights w[i] and profits p[i]
4. Input knapsack capacity W
5. Create a table dp[n+1][W+1]
6. Initialize first row and column with 0
7. For i = 1 to n
- For j = 1 to W
- If w[i] ≤ j
- dp[i][j] = max(p[i] + dp[i-1][j-w[i]], dp[i-1][j])
- Else
- dp[i][j] = dp[i-1][j]
8. Final result will be in dp[n][W]
9. Print maximum profit
10. Stop

Code:

```
#include <stdio.h>

int max(int a,int b)
{
    if(a>b)
        return a;
    return b;
}

int main()
{
    int n,W;
```

```
    printf("Enter number of items: ");
    scanf("%d",&n);

    int wi[n],vi[n];

    printf("Enter weights:\n");
    for(int i=0;i<n;i++)
        scanf("%d",&wi[i]);

    printf("Enter profits:\n");
    for(int i=0;i<n;i++)
        scanf("%d",&vi[i]);

    printf("Enter knapsack capacity: ");
    scanf("%d",&W);

    int dp[n+1][W+1];

    for(int i=0;i<=n;i++)
    {
        for(int w=0;w<=W;w++)
        {
            if(i==0 || w==0)
                dp[i][w]=0;

            else if(wi[i-1] <= w)
                dp[i][w] = max(vi[i-1] + dp[i-1][w-wi[i-1]],
dp[i-1][w]);

            else
                dp[i][w] = dp[i-1][w];
        }
    }

    printf("\nDP Table\n\n");

    printf("Item\\Cap ");
    for(int w=0;w<=W;w++)
        printf("%4d",w);

    printf("\n");

    for(int i=0;i<=n;i++)
    {
        if(i==0)
            printf("0       ");
        else
            printf("I%d(%d,%d) ",i,wi[i-1],vi[i-1]);
```

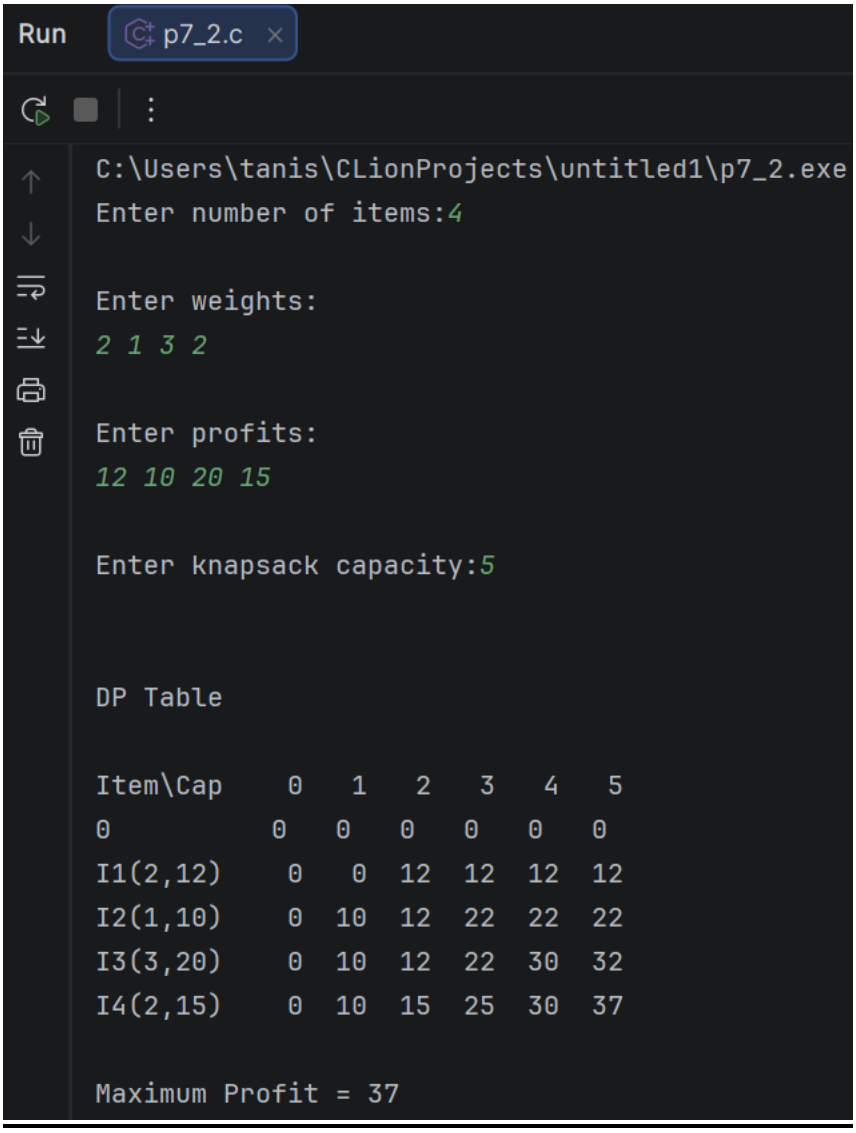
```
            for(int w=0;w<=W;w++)
                printf("%4d",dp[i][w]);

            printf("\n");
        }

        printf("\nMaximum Profit = %d\n",dp[n][W]);

        return 0;
    }
```

Output:



Practical 8

Title:

To solve Matrix Chain Multiplication using Dynamic Programming.

Objective:

To determine the **minimum number of scalar multiplications** required to multiply a chain of matrices.

Theory:

Matrix Chain Multiplication is a Dynamic Programming problem.
When multiplying multiple matrices, the order of multiplication affects the total number of operations.

Algorithm / Steps:

1. Start
2. Input number of matrices n
3. Input dimension array p[0…n]
   (Matrix Ai has dimension p[i−1]×p[i])
4. Create DP table m[n+1][n+1]
5. For i = 1 to n
   Set m[i][i] = 0
   (Cost of multiplying one matrix is 0)
6. For chain length L = 2 to n
7. For i = 1 to n − L + 1
8. Set j = i + L − 1
9. Initialize m[i][j] = ∞
10. For k = i to j − 1
11. Compute cost
12. cost=m[i][k]+m[k+1][j]+p[i−1]×p[k]×p[j]
13. If cost < m[i][j]
    Update m[i][j] = cost
14. Repeat until all chains are evaluated
15. Output m[1][n] as the minimum multiplication cost
16. Stop

Code:

```
#include <stdio.h>

 #define INF 999999

int main() {

int n;

printf("Enter number of matrices: ");
scanf("%d",&n);

int p[n+1];

printf("Enter dimensions array:\n");
for(int i=0;i<=n;i++)
 scanf("%d",&p[i]);

int m[n+1][n+1];

for(int i=1;i<=n;i++)
    m[i][i]=0;

for(int len=2;len<=n;len++)
{
    for(int i=1;i<=n-len+1;i++)
    {
        int j=i+len-1;
        m[i][j]=INF;

        for(int k=i;k<j;k++)
        {
            int q = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];

            if(q < m[i][j])
                m[i][j] = q;
        }
    }
}

printf("\nDP Table\n\n");

printf("m   ");
for(int j=1;j<=n;j++)
    printf("%8d",j);

printf("\n");

for(int i=1;i<=n;i++)
{
    printf("%d   ",i);

    for(int j=1;j<=n;j++)
    {
```

```
        if(i>j)
            printf("        ");
        else
            printf("%8d",m[i][j]);
    }

    printf("\n");
}

printf("\nMinimum multiplication cost = %d\n",m[1][n]);

return 0;

}
```

Outout:



Practical 9

Title:

To find the Longest Common Subsequence (LCS) of two strings using Dynamic Programming.

Objective:

To determine the **longest sequence of characters that appear in the same order in both strings**.

Theory:

The Longest Common Subsequence (LCS) problem finds the longest sequence that exists in both given strings while maintaining the order of characters.

Algorithm / Steps:

1. Start
2. Input two strings X and Y
3. Let m = length(X) and n = length(Y)
4. Create DP table L[m+1][n+1]
5. Initialize first row and column with 0
6. For i = 1 to m
7. For j = 1 to n
8. If X[i−1] == Y[j−1]
   L[i][j] = L[i−1][j−1] + 1
9. Else
   L[i][j] = max(L[i−1][j], L[i][j−1])
10. The value L[m][n] is the length of LCS
11. Stop

Code:

```
#include <stdio.h>
#include <string.h>

int max(int a,int b)
{
    return (a>b)?a:b;
}

int main()
{
    char X[100],Y[100];

    printf("Enter first string: ");
    scanf("%s",X);
```

```
    printf("Enter second string: ");
    scanf("%s",Y);

    int m=strlen(X);
    int n=strlen(Y);

    int L[m+1][n+1];

    for(int i=0;i<=m;i++)
    {
        for(int j=0;j<=n;j++)
        {
            if(i==0 || j==0)
                L[i][j]=0;

            else if(X[i-1]==Y[j-1])
                L[i][j]=L[i-1][j-1]+1;

            else
                L[i][j]=max(L[i-1][j],L[i][j-1]);
        }
    }

    printf("\nDP Table:\n");

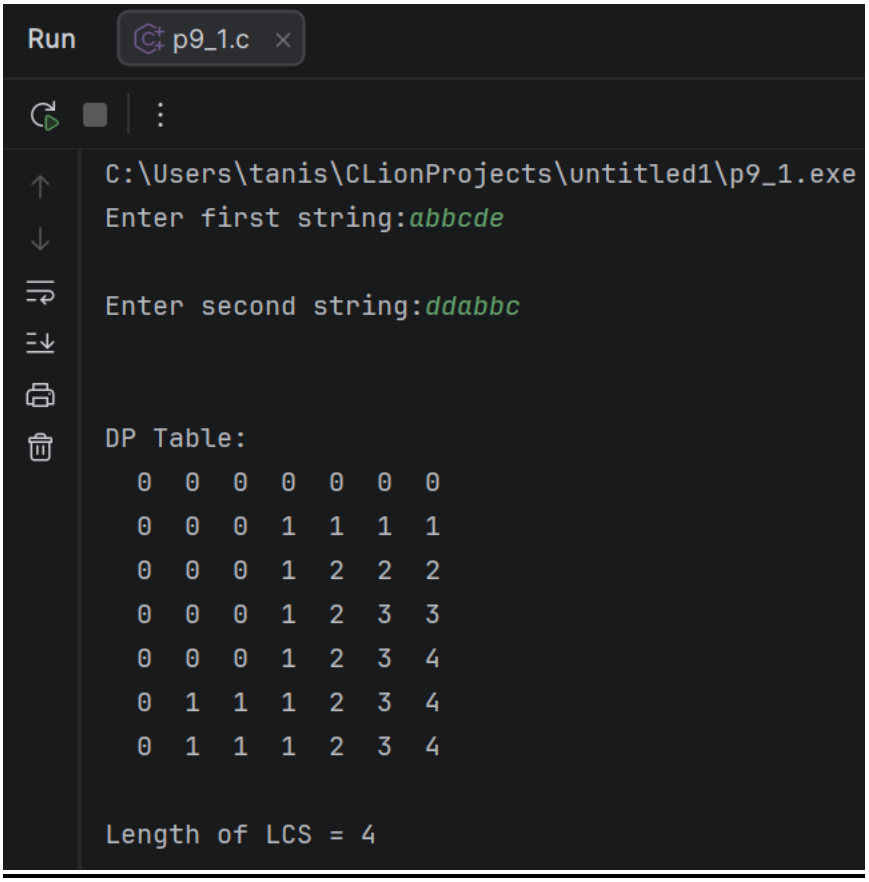
    for(int i=0;i<=m;i++)
    {
        for(int j=0;j<=n;j++)
            printf("%3d",L[i][j]);

        printf("\n");
    }

    printf("\nLength of LCS = %d\n",L[m][n]);

    return 0;
}
```

Output:


Run
p9_1.c
C:\Users\tanis\CLionProjects\untitled1\p9_1.exe
Enter first string:abbcde
Enter second string:ddabbc
DP Table:
0 0 0 0 0 0 0
0 0 0 1 1 1 1
0 0 0 1 2 2 2
0 0 0 1 2 3 3
0 0 0 1 2 3 4
0 1 1 1 2 3 4
0 1 1 1 2 3 4
Length of LCS = 4

Title:

To find the Shortest Common Supersequence using Dynamic Programming.

Objective:

To determine the **shortest string that contains both given strings as subsequences**.

Theory:

A Shortest Common Supersequence (SCS) is the smallest string that contains both input strings.

Algorithm / Steps:

1. Start
2. Input strings X and Y
3. Find LCS length using DP
4. Let m = length(X) and n = length(Y)
5. Compute
   - SCS Length = m + n − LCS
6. Print the result
7. Stop

Code:

```
#include <stdio.h>
#include <string.h>

int max(int a, int b)
{
    if(a>b)
        return a;
    else
        return b;
}

int main()
{
    char X[50], Y[50];
    int i,j;

    printf("Enter first string: ");
    scanf("%s",X);

    printf("\nEnter second string: ");
    scanf("%s",Y);

    int m = strlen(X);
    int n = strlen(Y);

    int L[m+1][n+1];

    // Filling DP table
```

```
    for(i=0;i<=m;i++)
    {
        for(j=0;j<=n;j++)
        {
            if(i==0 || j==0)
                L[i][j]=0;

            else if(X[i-1]==Y[j-1])
                L[i][j]=L[i-1][j-1]+1;

            else
                L[i][j]=max(L[i-1][j],L[i][j-1]);
        }
    }

    printf("\nDP Table:\n\n");

    printf("    ");
    for(j=0;j<n;j++)
        printf(" %c ",Y[j]);

    printf("\n");

    for(i=0;i<=m;i++)
    {
        if(i==0)
            printf("  ");
        else
            printf("%c ",X[i-1]);

        for(j=0;j<=n;j++)
        {
            printf("%2d ",L[i][j]);
        }

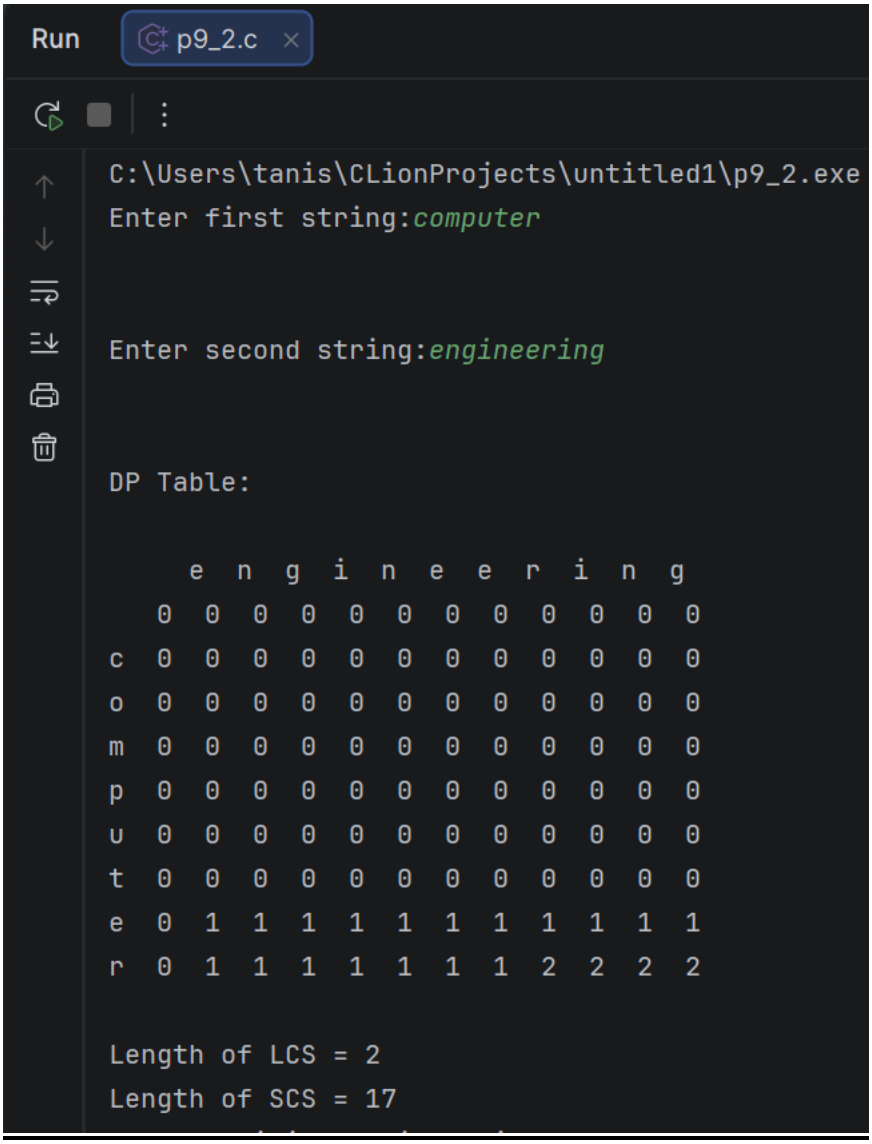
        printf("\n");
    }

    int lcs = L[m][n];
    int scs = m + n - lcs;

    printf("\nLength of LCS = %d",lcs);
    printf("\nLength of SCS = %d",scs);

    return 0;
}
```

Output:


Run
p9_2.c
C:\Users\tanis\CLionProjects\untitled1\p9_2.exe
Enter first string:computer
Enter second string:engineering
DP Table:
e n g i n e e r i n g
0 0 0 0 0 0 0 0 0 0 0 0
c 0 0 0 0 0 0 0 0 0 0 0 0
o 0 0 0 0 0 0 0 0 0 0 0 0
m 0 0 0 0 0 0 0 0 0 0 0 0
p 0 0 0 0 0 0 0 0 0 0 0 0
u 0 0 0 0 0 0 0 0 0 0 0 0
t 0 0 0 0 0 0 0 0 0 0 0 0
e 0 1 1 1 1 1 1 1 1 1 1 1
r 0 1 1 1 1 1 1 1 2 2 2 2
Length of LCS = 2
Length of SCS = 17

Practical 4

Title:

Solve the **Make a Change Problem using Greedy Strategy**

Objective:

To find the minimum number of coins required to make a given amount using a greedy approach.

Theory:

The Make a Change Problem in Design and Analysis of Algorithms uses a greedy approach to find the minimum number of coins for a given amount. The largest possible coin is selected at each step until the amount becomes zero. This method is efficient for standard coin systems.

Algorithm / Steps:

1. Start
2. Input amount `A`
3. Define coin denominations in descending order
4. For each coin:
5. While `A >= coin`
    5.1. Print coin
    5.2. Reduce `A = A - coin`
6. Stop when `A = 0`
7. End

Code:

```
#include <stdio.h>

int main() {
    int coins[] = {50, 20, 10, 5, 2, 1};
    int n = 6;
    int amount, i;

    printf("Enter amount: ");
    scanf("%d", &amount);

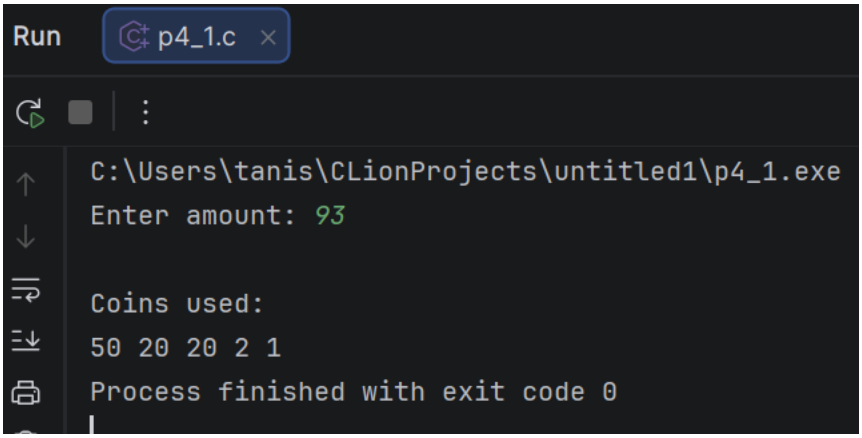
    printf("Coins used:\n");

    for(i = 0; i < n; i++) {
        while(amount >= coins[i]) {
            printf("%d ", coins[i]);
            amount -= coins[i];
        }
```

```
    }

    return 0;
}
```

Output:



Title:

Solve the **Activity Selection Problem using Greedy Strategy**

Objective:

To select the maximum number of non-overlapping activities.

Theory:

The Activity Selection Problem in Design and Analysis of Algorithms selects the maximum number of non-overlapping activities. Activities are sorted by finish time, and the earliest finishing activity is chosen first. This ensures an optimal solution using the greedy method.

Algorithm / Steps:

1. Start
2. Input number of activities `n`
3. Input start and finish times
4. Sort activities based on finish time
5. Select first activity
6. For each next activity:
   If `start >= last_finish`, select it
7. Print selected activities
8. End

Code:

(A) Without Structure

```
#include <stdio.h>

int main() {
    int n, i, j;

    printf("Enter number of activities: ");
    scanf("%d", &n);

    int start[n], finish[n];

    printf("Enter start times:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &start[i]);

    printf("Enter finish times:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &finish[i]);

    // Sorting based on finish time
    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
```

```
            if(finish[j] > finish[j + 1]) {
                int temp = finish[j];
                finish[j] = finish[j + 1];
                finish[j + 1] = temp;

                temp = start[j];
                start[j] = start[j + 1];
                start[j + 1] = temp;
            }
        }
    }

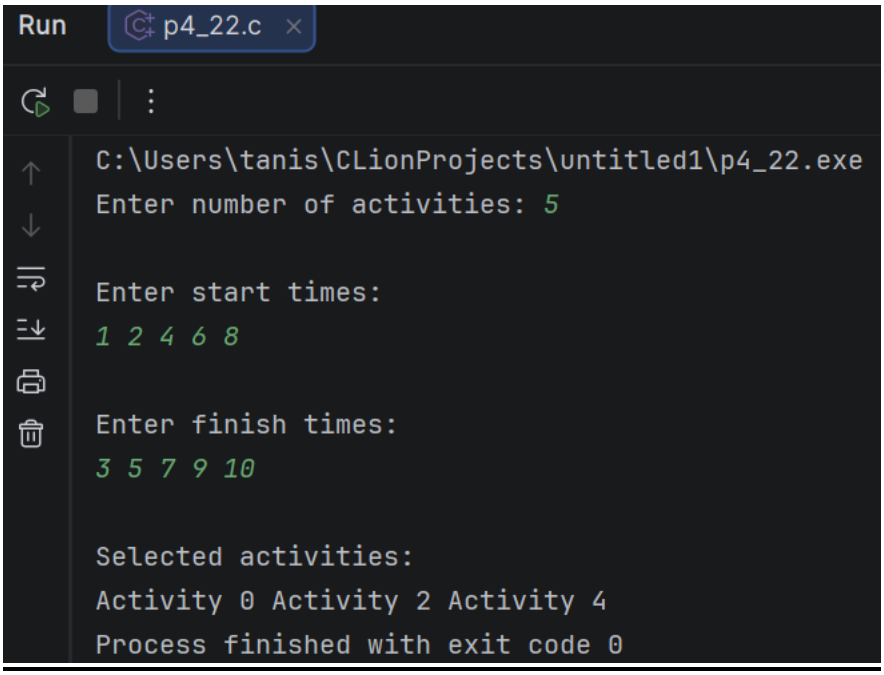
    printf("Selected activities:\n");

    i = 0;
    printf("Activity %d ", i);

    for(j = 1; j < n; j++) {
        if(start[j] >= finish[i]) {
            printf("Activity %d ", j);
            i = j;
        }
    }

    return 0;
}
```

Output:



(B) With Structure

```
#include <stdio.h>

struct Activity {
    int start;
    int finish;
};

int main() {
    int n, i, j;

    printf("Enter number of activities: ");
    scanf("%d", &n);

    struct Activity a[n];

    printf("Enter start and finish times:\n");
    for(i = 0; i < n; i++) {
        scanf("%d %d", &a[i].start, &a[i].finish);
    }

    // Sorting based on finish time
    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
            if(a[j].finish > a[j + 1].finish) {
                struct Activity temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

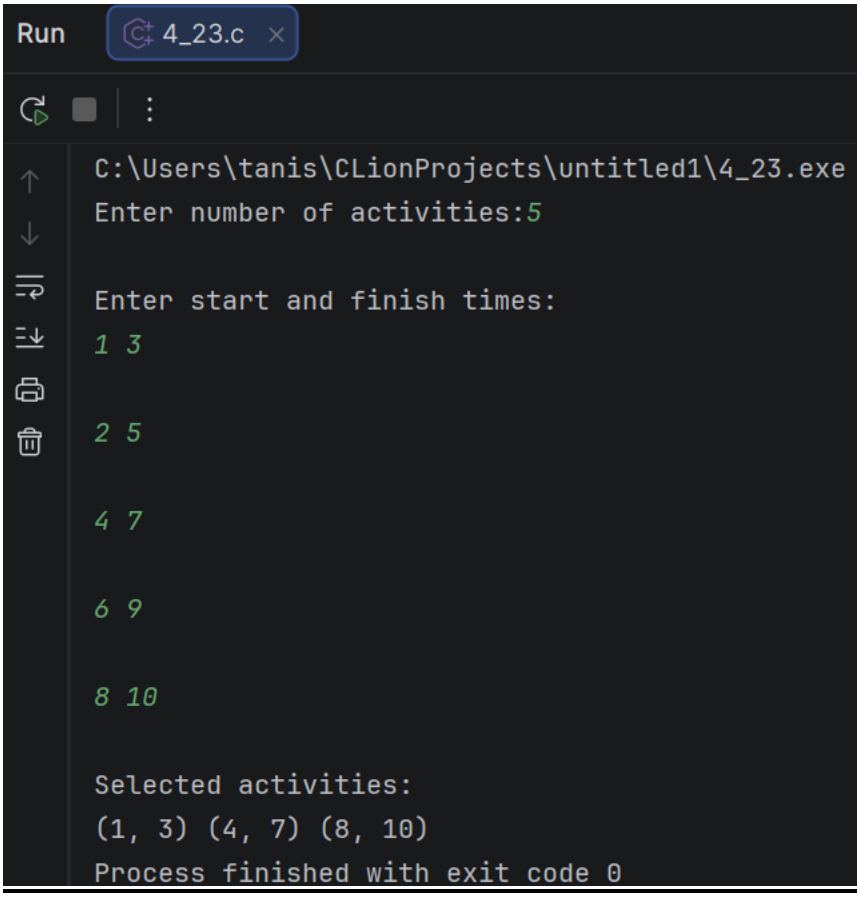
    printf("Selected activities:\n");

    i = 0;
    printf("(%d, %d) ", a[i].start, a[i].finish);

    for(j = 1; j < n; j++) {
        if(a[j].start >= a[i].finish) {
            printf("(%d, %d) ", a[j].start, a[j].finish);
            i = j;
        }
    }

    return 0;
}
```

Output:


Run
4_23.c
C:\Users\tanis\CLionProjects\untitled1\4_23.exe
Enter number of activities:5
Enter start and finish times:
1 3
2 5
4 7
6 9
8 10
Selected activities:
(1, 3) (4, 7) (8, 10)
Process finished with exit code 0

Practical 5

Title:

Solve the **Fractional Knapsack Problem using Greedy Strategy**

Objective:

To maximize profit by selecting items based on profit/weight ratio.

Theory:

The Fractional Knapsack Problem in Design and Analysis of Algorithms uses a greedy approach to maximize profit. Items are selected based on highest profit-to-weight ratio. Fractions of items are allowed.

Algorithm / Steps:

1. Input weights, profits, capacity
2. Compute profit/weight ratio
3. Sort items in decreasing ratio
4. Pick items fully or fractionally
5. Stop when capacity is full

Code:

(A) Without Structure

```
#include <stdio.h>

int main() {
    int n, i, j;
    float weight[10], profit[10], ratio[10], capacity;

    printf("Enter number of items: ");
    scanf("%d", &n);

    printf("Enter weights:\n");
    for(i = 0; i < n; i++) scanf("%f", &weight[i]);

    printf("Enter profits:\n");
    for(i = 0; i < n; i++) scanf("%f", &profit[i]);

    printf("Enter capacity: ");
    scanf("%f", &capacity);

    for(i = 0; i < n; i++)
```

```
        ratio[i] = profit[i] / weight[i];

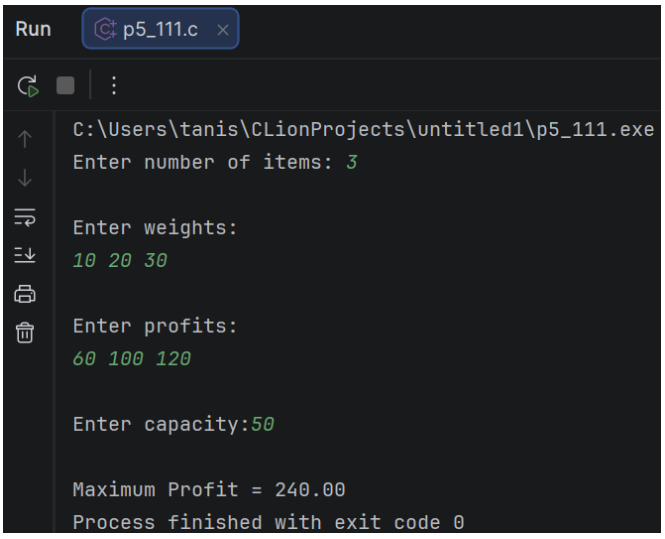
    // Sorting
    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
            if(ratio[j] < ratio[j+1]) {
                float temp;
                temp = ratio[j]; ratio[j] = ratio[j+1]; ratio[j+1] = temp;
                temp = weight[j]; weight[j] = weight[j+1]; weight[j+1] = temp;
                temp = profit[j]; profit[j] = profit[j+1]; profit[j+1] = temp;
            }
        }
    }

    float totalProfit = 0;

    for(i = 0; i < n; i++) {
        if(capacity >= weight[i]) {
            totalProfit += profit[i];
            capacity -= weight[i];
        } else {
            totalProfit += ratio[i] * capacity;
            break;
        }
    }

    printf("Maximum Profit = %.2f", totalProfit);
    return 0;
}
```

Output:



(B) With Structure

```
#include <stdio.h>

struct Item {
    float weight, profit, ratio;
```

```
};

int main() {
    int n, i, j;
    struct Item a[10];
    float capacity;

    printf("Enter number of items: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        scanf("%f %f", &a[i].weight, &a[i].profit);
        a[i].ratio = a[i].profit / a[i].weight;
    }

    printf("Enter capacity: ");
    scanf("%f", &capacity);

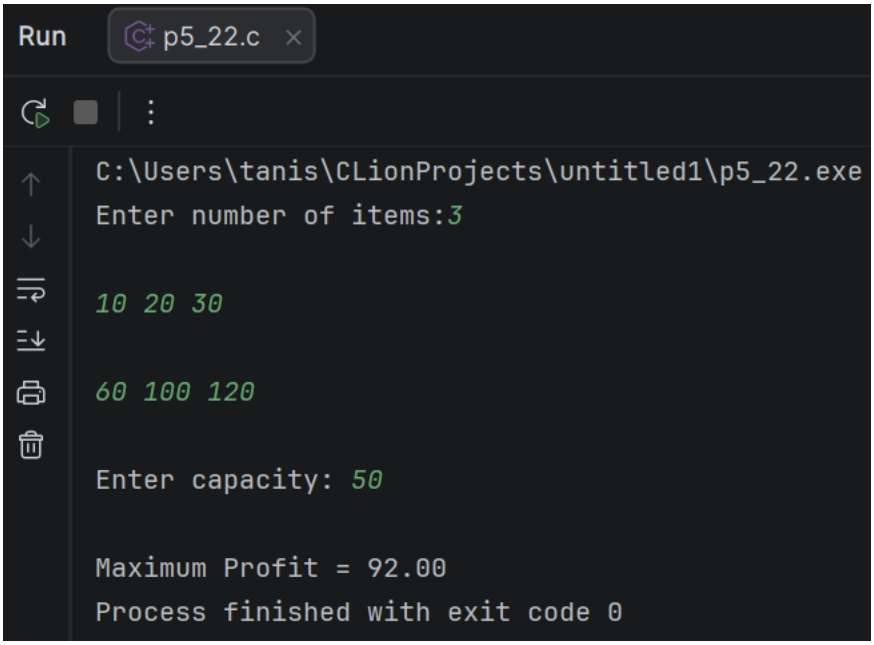
    // Sorting
    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
            if(a[j].ratio < a[j+1].ratio) {
                struct Item temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }

    float totalProfit = 0;

    for(i = 0; i < n; i++) {
        if(capacity >= a[i].weight) {
            totalProfit += a[i].profit;
            capacity -= a[i].weight;
        } else {
            totalProfit += a[i].ratio * capacity;
            break;
        }
    }

    printf("Maximum Profit = %.2f", totalProfit);
    return 0;
}
```

Output:


Run
p5_22.c
C:\Users\tanis\CLionProjects\untitled1\p5_22.exe
Enter number of items:3
10 20 30
60 100 120
Enter capacity: 50
Maximum Profit = 92.00
Process finished with exit code 0

Title:

Solve the **Job Scheduling Problem using Greedy Strategy**

Objective:

To maximize profit by scheduling jobs within deadlines.

Theory:

The Job Scheduling Problem in Design and Analysis of Algorithms selects jobs to maximize profit within given deadlines. Jobs are sorted by profit, and each job is scheduled in the latest available slot before its deadline.

Algorithm / Steps:

1. Input jobs with deadline and profit
2. Sort jobs by decreasing profit
3. Assign each job to latest free slot
4. Repeat for all jobs

Code:

(A) Without Structure

```
#include <stdio.h>

int main() {
    int n, i, j;

    printf("Enter number of jobs: ");
    scanf("%d", &n);

    int deadline[n], profit[n];

    for(i = 0; i < n; i++) {
        scanf("%d %d", &deadline[i], &profit[i]);
    }

    // Sort by profit
    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
            if(profit[j] < profit[j+1]) {
                int temp = profit[j];
                profit[j] = profit[j+1];
                profit[j+1] = temp;

                temp = deadline[j];
```

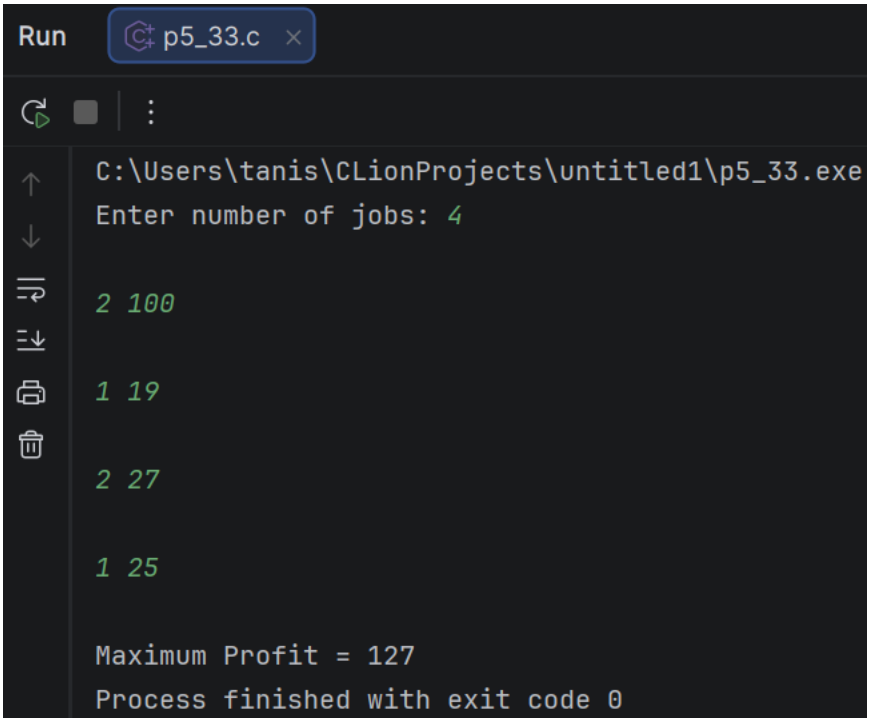
```
                deadline[j] = deadline[j+1];
                deadline[j+1] = temp;
            }
        }
    }

    int slot[10] = {0};
    int totalProfit = 0;

    for(i = 0; i < n; i++) {
        for(j = deadline[i] - 1; j >= 0; j--) {
            if(slot[j] == 0) {
                slot[j] = 1;
                totalProfit += profit[i];
                break;
            }
        }
    }

    printf("Maximum Profit = %d", totalProfit);
    return 0;
}
```

Output:



(B) With Structure

```
#include <stdio.h>

struct Job {
    int id, deadline, profit;
};

int main() {
    int n, i, j;

    printf("Enter number of jobs: ");
    scanf("%d", &n);

    struct Job a[10];

    for(i = 0; i < n; i++) {
        scanf("%d %d", &a[i].deadline, &a[i].profit);
        a[i].id = i + 1;
    }

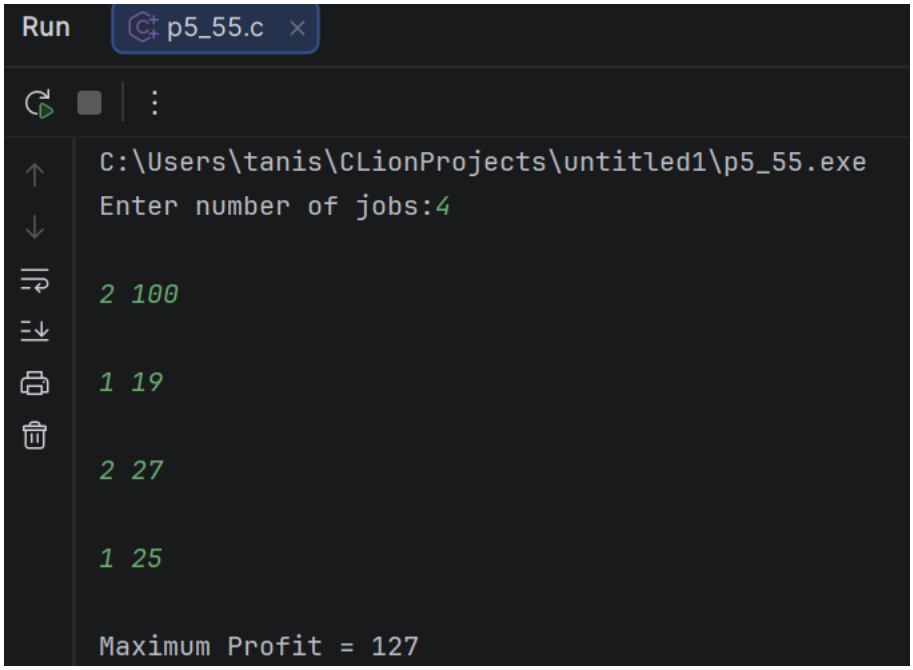
    // Sort by profit
    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
            if(a[j].profit < a[j+1].profit) {
                struct Job temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }

    int slot[10] = {0};
    int totalProfit = 0;

    for(i = 0; i < n; i++) {
        for(j = a[i].deadline - 1; j >= 0; j--) {
            if(slot[j] == 0) {
                slot[j] = 1;
                totalProfit += a[i].profit;
                break;
            }
        }
    }

    printf("Maximum Profit = %d", totalProfit);
    return 0;
}
```

Output:


Run
p5_55.c
C:\Users\tanis\CLionProjects\untitled1\p5_55.exe
Enter number of jobs:4
2 100
1 19
2 27
1 25
Maximum Profit = 127

Practical 6

Title:

Minimum Spanning Tree using **Prim's Algorithm**

Objective:

To find MST of a graph using greedy approach.

Theory:

Prim's Algorithm in Design and Analysis of Algorithms builds the MST by selecting the minimum weight edge that connects a visited vertex to an unvisited vertex. It starts from any vertex and grows the tree step by step. It always chooses the smallest edge to avoid cycles.

Algorithm / Steps:

1. Start from any vertex
2. Mark it visited
3. Select minimum edge connecting visited → unvisited
4. Add edge to MST
5. Repeat until all vertices are included

Code:

```
#include <stdio.h>
#define INF 999

int main() {
    int n, i, j, min, u, v;
    int visited[10] = {0};
    int cost[10][10];

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    visited[0] = 1;
    int edges = 0, totalCost = 0;

    while(edges < n - 1) {
        min = INF;
```

```
        for(i = 0; i < n; i++) {
            if(visited[i]) {
                for(j = 0; j < n; j++) {
                    if(!visited[j] && cost[i][j] < min) {
                        min = cost[i][j];
                        u = i;
                        v = j;
                    }
                }
            }
        }

        printf("Edge: %d - %d = %d\n", u, v, min);
        totalCost += min;
        visited[v] = 1;
        edges++;
    }

    printf("Total Cost = %d", totalCost);
    return 0;
}
```

Output:



Title:

Minimum Spanning Tree using **Kruskal's Algorithm**

Objective:

To find MST by selecting minimum edges without forming cycles.

Theory:

Kruskal's Algorithm in Design and Analysis of Algorithms constructs MST by sorting all edges in increasing order of weight. It selects edges one by one, ensuring no cycle is formed. It uses union-find method for cycle detection.

Algorithm / Steps:

- Sort edges in increasing order
- Pick smallest edge
- Check cycle
- If no cycle → include in MST
- Repeat until n-1 edges

Code:

```
#include <stdio.h>

int parent[10];

int find(int i) {
    while(parent[i])
        i = parent[i];
    return i;
}

int unionSet(int i, int j) {
    if(i != j) {
        parent[j] = i;
        return 1;
    }
    return 0;
}

int main() {
    int n, i, j, min, a, b, u, v;
    int cost[10][10];

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost matrix:\n");
    for(i = 0; i < n; i++)
```

```
        for(j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    int edges = 0, totalCost = 0;

    while(edges < n - 1) {
        min = 999;

        for(i = 0; i < n; i++) {
            for(j = 0; j < n; j++) {
                if(cost[i][j] < min) {
                    min = cost[i][j];
                    a = u = i;
                    b = v = j;
                }
            }
        }

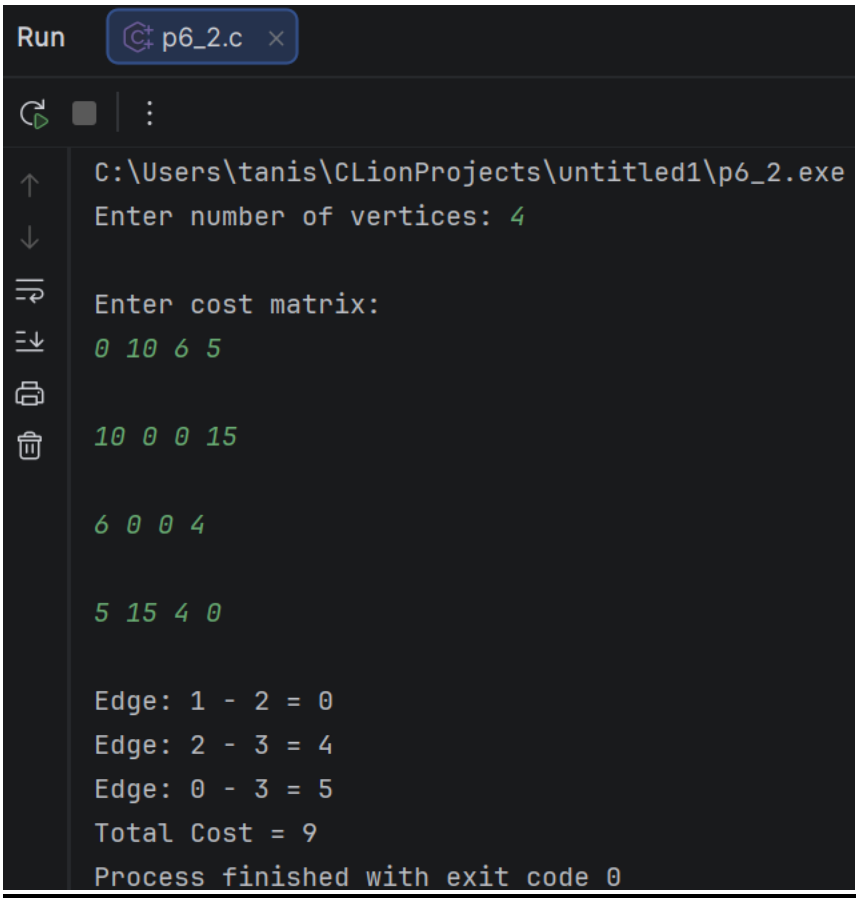
        u = find(u);
        v = find(v);

        if(unionSet(u, v)) {
            printf("Edge: %d - %d = %d\n", a, b, min);
            totalCost += min;
            edges++;
        }

        cost[a][b] = cost[b][a] = 999;
    }

    printf("Total Cost = %d", totalCost);
    return 0;
}
```

Output:


Run
p6_2.c
C:\Users\tanis\CLionProjects\untitled1\p6_2.exe
Enter number of vertices: 4
Enter cost matrix:
0 10 6 5
10 0 0 15
6 0 0 4
5 15 4 0
Edge: 1 - 2 = 0
Edge: 2 - 3 = 4
Edge: 0 - 3 = 5
Total Cost = 9
Process finished with exit code 0

Practical 10

Title:

Solve **4-Queen Problem using Backtracking**

Objective:

To place 4 queens on a 4×4 chessboard such that no two queens attack each other.

Theory:

The N-Queen Problem in Design and Analysis of Algorithms uses backtracking to place queens safely. A queen must not share the same row, column, or diagonal. If a conflict occurs, we backtrack and try another position.

Algorithm / Steps:

1. Start from first row
2. Place queen in a safe column
3. Move to next row
4. If no safe position → backtrack
5. Repeat until solution found

Code:

4-Queen

```
#include <stdio.h>

int x[10], count = 0;

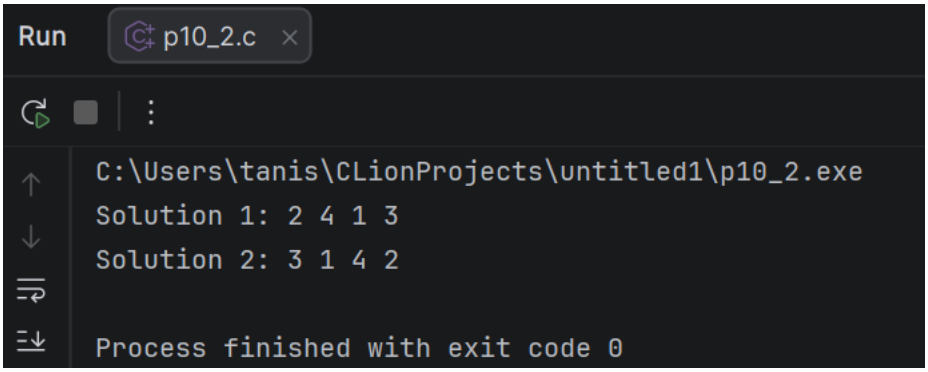
int place(int k, int i) {
    for(int j = 1; j < k; j++) {
        if(x[j] == i || abs(x[j] - i) == abs(j - k))
            return 0;
    }
    return 1;
}

void nqueen(int k, int n) {
    for(int i = 1; i <= n; i++) {
        if(place(k, i)) {
            x[k] = i;
            if(k == n) {
```

```
                count++;
                printf("Solution %d: ", count);
                for(int j = 1; j <= n; j++)
                    printf("%d ", x[j]);
                printf("\n");
            } else {
                nqueen(k + 1, n);
            }
        }
    }
}

int main() {
    nqueen(1, 4);
    return 0;
}
```

Output:



8-Queen

```
#include <stdio.h>
#include <stdlib.h>

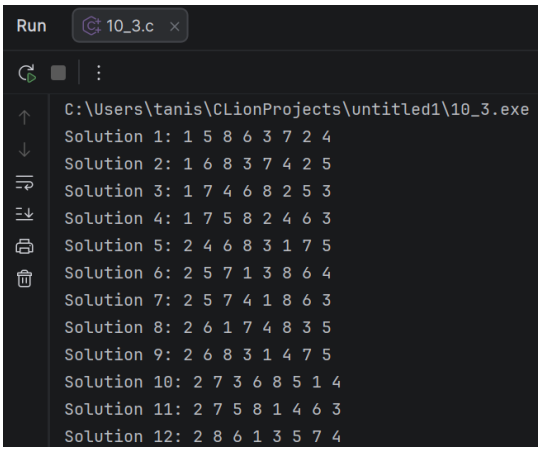
int x[20], count = 0;

int place(int k, int i) {
    for(int j = 1; j < k; j++) {
        if(x[j] == i || abs(x[j] - i) == abs(j - k))
            return 0;
    }
    return 1;
}

void nqueen(int k, int n) {
    for(int i = 1; i <= n; i++) {
        if(place(k, i)) {
            x[k] = i;
            if(k == n) {
                count++;
                printf("Solution %d: ", count);
                for(int j = 1; j <= n; j++)
                    printf("%d ", x[j]);
                printf("\n");
            } else {
                nqueen(k + 1, n);
            }
        }
    }
}

int main() {
    nqueen(1, 8);
    return 0;
}
```

Output:



Practical 3

Title:

Perform **Merge Sort and measure execution time**

Objective:

To sort elements using Merge Sort and calculate execution time for large input.

Theory:

Merge Sort in Design and Analysis of Algorithms is a divide-and-conquer algorithm. It divides the array into halves, sorts them recursively, and merges them. Its time complexity is **O(n log n)**.

Algorithm / Steps:

1. Divide array into two halves
2. Recursively sort both halves
3. Merge sorted halves
4. Repeat until sorted

Code:

```
#include <stdio.h>
#include <time.h>

void merge(int arr[], int l, int m, int r) {
    int i=l, j=m+1, k=0, temp[100000];

    while(i<=m && j<=r) {
        if(arr[i] < arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while(i<=m) temp[k++] = arr[i++];
    while(j<=r) temp[k++] = arr[j++];

    for(i=l, k=0; i<=r; i++, k++)
        arr[i] = temp[k];
}

void mergeSort(int arr[], int l, int r) {
    if(l < r) {
        int m = (l + r) / 2;
```

```
        mergeSort(arr, l, m);
        mergeSort(arr, m+1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    int n = 1000, arr[100000];
    clock_t start, end;

    for(int i=0;i<n;i++)
        arr[i] = n - i;

    start = clock();
    mergeSort(arr, 0, n-1);
    end = clock();

    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Execution Time (Merge Sort): %f seconds", time_taken);
    return 0;
}
```

Output:



Title:

Perform **Quick Sort and measure execution time**

Objective:

To sort elements using Quick Sort and calculate execution time.

Theory:

Quick Sort in Design and Analysis of Algorithms is a divide-and-conquer algorithm. It selects a pivot and partitions the array into two parts. Average time complexity is **O(n log n)**.

Algorithm / Steps:

1. Choose pivot element
2. Partition array
3. Recursively sort left and right parts
4. Repeat until sorted

Code:

```
#include <stdio.h>
#include <time.h>

void quickSort(int arr[], int low, int high) {
    if(low < high) {
        int pivot = arr[low];
        int i = low + 1;
        int j = high;
        int temp;

        while(i <= j) {
            while(arr[i] <= pivot && i <= high) i++;
            while(arr[j] > pivot) j--;

            if(i < j) {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        temp = arr[low];
        arr[low] = arr[j];
        arr[j] = temp;

        quickSort(arr, low, j-1);
        quickSort(arr, j+1, high);
    }
}
```

```
int main() {
    int n = 1000, arr[100000];
    clock_t start, end;

    for(int i=0;i<n;i++)
        arr[i] = n - i;

    start = clock();
    quickSort(arr, 0, n-1);
    end = clock();

    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Execution Time (Quick Sort): %f seconds", time_taken);
    return 0;
}
```

Output:

